

The present work was submitted to the Institute for Data Science in Mechanical Engineering.
Diese Arbeit wurde vorgelegt am Institut für Data Science im Maschinenbau.

Model-based Multi-agent Reinforcement Learning via Uncertainty Quantification

Project Thesis

Projektarbeit

Presented by / Vorgelegt von

Yachen Tian
Matr.Nr.: 463238

Supervised by / Betreut von	Alexander Gräfe M.Sc. (Institute for Data Science in Mechanical Engineering)
1st Examiner / 1. Prüfer	Univ.-Prof. Dr. sc. Sebastian Trimpe (Institute for Data Science in Mechanical Engineering)
2nd Examiner / 2. Prüfer	Dr. Friedrich Solowjow (Institute for Data Science in Mechanical Engineering)

Aachen, March 20, 2026

Model-based Multi-agent Reinforcement Learning via Uncertainty Quantification

Yachen Tian^{1,2}

¹*RWTH Aachen University*

²*Institute for Data Science in Mechanical Engineering*

Abstract: Model-based approaches are attractive in multi-agent reinforcement learning because they can improve sample efficiency by using learned models to generate additional training data. However, obtaining reliable model rollouts in multi-agent settings remains challenging, since errors may arise from both inaccurate opponent predictions and unreliable dynamics models, while communication with other agents also incurs additional cost. To address this issue, we propose an uncertainty-aware model-based multi-agent reinforcement learning method that uses opponent-model uncertainty to decide when communication is needed and dynamics-model uncertainty to decide when model rollouts should be terminated. We evaluate the proposed method on two JaxMARL environments, and the results show better performance on the Spread environment and comparable performance on the FACMAC environment compared with AORPO. Overall, these results indicate that uncertainty-aware communication and rollout control can improve the reliability of model-based multi-agent learning.

Keywords: *multi-agent reinforcement learning, model-based reinforcement learning, uncertainty-aware communication, adaptive rollouts, opponent modeling*

Eidesstattliche Erklärung

Hiermit erkläre ich an Eides statt, dass ich die vorliegende Arbeit ohne unzulässige Hilfe Dritter und ohne die Benutzung anderer als der angegebenen Hilfsmittel selbstständig verfasst habe; die aus anderen Quellen direkt oder indirekt übernommenen Daten und Konzepte sind unter Angabe des Literaturzitats gekennzeichnet.

Ich habe des Weiteren zur Kenntnis genommen, dass die Kennzeichnungspflicht in gleicher Weise für Hilfsmittel basierend auf generativer Künstlicher Intelligenz (KI) gilt. Die Pflicht zur expliziten Kennzeichnung entfällt lediglich, wenn der Einsatz generativer KI sich nicht auf den wissenschaftlichen Inhalt der Arbeit auswirkt (z.B. auf Grammatik-, Stil-, Rechtschreibprüfung, Übersetzungsprogramme, automatisierte Codekorrektur bzw. Codegenerierung die keinen Einfluss auf den wissenschaftlichen Inhalt hat). Ich übernehme die volle Verantwortung für die Integrität und Korrektheit aller Inhalte der Arbeit inklusiver der durch KI generierten Inhalte.

Aachen, den March 20, 2026

Contents

1	Introduction	4
1.1	Thesis Outline	4
2	Related Work	5
2.1	Model-Based Reinforcement Learning and Uncertainty-Aware Rollouts	5
2.2	Multi-Agent Reinforcement Learning and Opponent Modeling	6
2.3	Selective Communication in Model-Based Multi-Agent Rollouts	6
2.4	Summary	6
3	Method	7
3.1	Problem Formulation	7
3.2	Overall Framework	7
3.3	Opponent Modeling	8
3.4	Environment Dynamics and Reward Model	9
3.5	Uncertainty-Aware Adaptive Rollouts	11
3.6	Policy Optimization	13
4	Results	13
4.1	Experimental Setup	14
4.2	Main Results	14
4.3	Ablation Study	15
4.4	Discussion	16
5	Conclusion	16
A	Hyperparameters	18

1 Introduction

Multi-agent reinforcement learning (MARL) provides a general framework for sequential decision-making problems with multiple interacting agents and has been widely studied in both cooperative and competitive settings (Du and Ding, 2020). Model-based reinforcement learning is attractive because it can improve sample efficiency by learning an environment dynamics model and generating simulated experience for policy optimization (Janner et al., 2019; Sutton, 1990). However, model-based learning becomes more difficult in multi-agent settings. Future transitions depend not only on the environment dynamics but also on the behaviors of other agents. As a result, reliable model rollouts are harder to obtain in MARL than in single-agent problems. This makes rollout quality a key issue for effective model-based multi-agent learning.

A major difficulty in model-based MARL is that rollout errors can come from two different sources. First, the agent must estimate the actions of other agents during model rollouts. If these opponent actions are predicted inaccurately, the resulting joint action can be unreliable, which directly degrades the quality of simulated transitions (He et al., 2016). Second, even when the action inputs are accurate, the learned dynamics model may become less reliable as the rollout horizon increases. This can lead to compounding prediction errors and low-quality simulated data (Chua et al., 2018; Janner et al., 2019). Communication can help reduce the error caused by inaccurate opponent predictions, but it also introduces additional cost. Therefore, an effective method should decide both when communication is needed and when model rollouts should stop.

Recent work has explored model-based MARL with opponent modeling and adaptive rollouts (Zhang et al., 2021). As the most closely related framework, AORPO schedules communication according to the predicted relative error of each opponent model, which determines from which rollout step communication starts. However, this mechanism is not directly tied to the current reliability of opponent-action prediction, and it implicitly assumes that earlier rollout steps need less communication than later ones. Moreover, the rollout length of environment transitions is preplanned and gradually increased with training epochs. As a result, rollout termination is not directly determined by the current reliability of the dynamics model. Inspired by recent uncertainty-aware rollout control in model-based learning (Frauenknecht et al., 2024, 2025), we address these limitations by incorporating uncertainty quantification into both the opponent models and the dynamics model. Specifically, opponent-model uncertainty is used to decide when communication is needed, while dynamics-model uncertainty is used to decide when model rollouts should be terminated. In this way, the proposed method improves rollout reliability, supports event-triggered communication, and provides better simulated data for policy learning.

We evaluate the proposed method on two JaxMARL environments. The results show clear improvement on `mpe_simple_spread_v3` and comparable performance on `mpe_simple_facmac_3a_v1`, indicating better or comparable overall performance across tasks. The main contributions of this work are as follows:

1. We propose an uncertainty-aware model-based MARL framework for opponent modeling and adaptive policy learning.
2. We incorporate uncertainty quantification into both the opponent models and the dynamics model, using these uncertainty estimates for communication control and rollout termination respectively.
3. We validate the proposed method on two JaxMARL environments and demonstrate better or comparable performance across tasks.

1.1 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 2 reviews related work on model-based reinforcement learning, multi-agent reinforcement learning, opponent modeling, and selective communication in model-based multi-agent rollouts. Chapter 3 presents the proposed method, including the problem formulation, overall framework, opponent modeling, environment dynamics and reward modeling, uncertainty-aware adaptive rollouts, and policy optimization. Chapter 4 describes the experimental setup and reports the main results, ablation studies, and discussion. Finally, Chapter 5 concludes this thesis and discusses possible directions for future work.

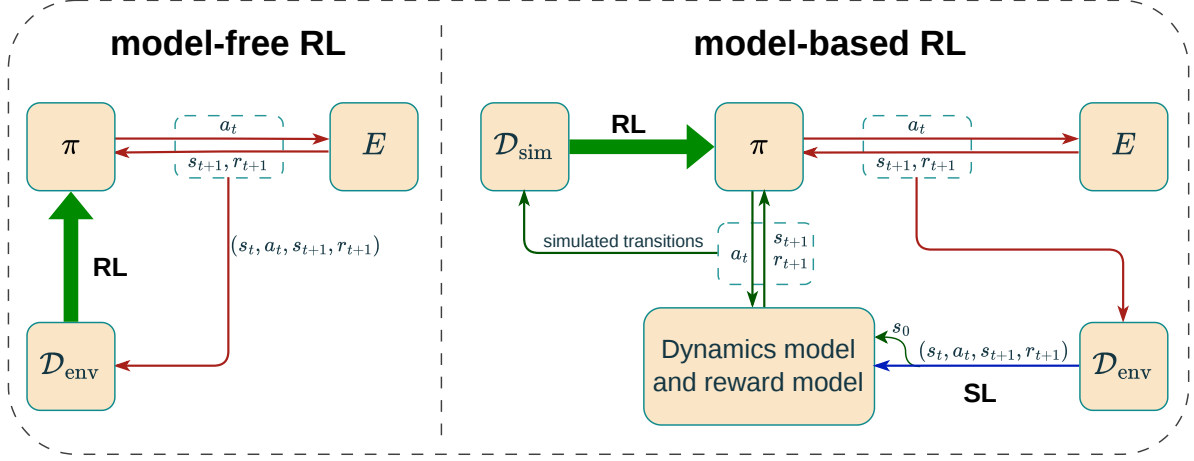


Figure 1. Comparison between model-free reinforcement learning and model-based reinforcement learning. In model-free RL, policy learning relies only on real environment transitions. In model-based RL, a learned dynamics and reward model generates simulated transitions to improve data efficiency.

2 Related Work

This chapter reviews the research directions most relevant to the proposed method. We first discuss model-based reinforcement learning and uncertainty-aware rollout control, which provide the basis for improving rollout reliability. We then review multi-agent reinforcement learning and opponent modeling, followed by selective communication in model-based multi-agent rollouts. Together, these lines of work motivate a unified framework that considers both opponent-prediction reliability and dynamics-model reliability in model-based multi-agent learning.

2.1 Model-Based Reinforcement Learning and Uncertainty-Aware Rollouts

Model-based reinforcement learning (MBRL) improves sample efficiency by learning a model of the environment dynamics and reward function, which can then be used to generate simulated transitions for policy optimization (Deisenroth and Rasmussen, 2011; Janner et al., 2019; Moerland et al., 2023; Sutton, 1990). Unlike model-free reinforcement learning, which relies solely on real environment interactions, MBRL can leverage learned models to perform rollouts from previously observed states, as illustrated in Fig. 1.

A central challenge in MBRL is that learned models are usually imperfect. As model rollouts become longer, prediction errors may accumulate over time and lead to unreliable simulated data (Talvitie, 2014). This creates a fundamental trade-off between rollout length and rollout fidelity: longer rollouts may provide more simulated training data, but they also increase the risk of model bias. As a result, an important question in MBRL is how to use learned models effectively without relying on them in regions where they are no longer trustworthy (Janner et al., 2019).

Existing work addresses this problem in several ways. Chua et al. (2018) improve robustness and quantify predictive uncertainty using ensembles of probabilistic dynamics models. Janner et al. (2019) reduce model bias by restricting the rollout horizon during policy optimization. More recent work further explores uncertainty-aware rollout control, where model uncertainty is used to adapt rollout usage or terminate rollouts when the learned dynamics become unreliable (Frauenknecht et al., 2024, 2025). These methods move beyond fixed-horizon rollouts by allowing model usage to depend on the estimated reliability of the learned dynamics. However, they are mainly developed for single-agent settings, where rollout quality depends primarily on the learned dynamics model. In multi-agent settings, future transitions additionally depend on the actions of other agents, introducing an additional source of uncertainty during model rollouts.

2.2 Multi-Agent Reinforcement Learning and Opponent Modeling

In multi-agent reinforcement learning (MARL), multiple agents interact simultaneously in a shared environment (Du and Ding, 2020; Littman, 1994). From the perspective of a single agent, the environment is effectively non-stationary because future transitions depend not only on the environment dynamics but also on the evolving policies of other agents (Hernandez-Leal et al., 2017). This makes stable and sample-efficient learning substantially more difficult than in single-agent reinforcement learning.

A common way to address this challenge is opponent modeling, where an agent explicitly learns to predict the behavior of other agents instead of treating them as part of an unknown environment. Prior work has shown that estimating opponent actions can improve decision-making in multi-agent settings (He et al., 2016; Raileanu et al., 2018). This is especially important in model-based multi-agent reinforcement learning, where rollout generation depends not only on a learned dynamics model but also on predicted opponent actions.

Model-based MARL therefore extends standard MBRL by learning both environment dynamics and models of other agents. Prior work has incorporated opponent modeling into model-based rollouts and has shown that selective communication can help reduce the cost of acquiring opponent actions during planning (Zhang et al., 2021). However, maintaining reliable opponent predictions during rollouts remains difficult, especially when prediction quality depends on the current rollout state. This motivates methods that explicitly assess opponent-prediction reliability during rollouts rather than relying on fixed communication schedules.

2.3 Selective Communication in Model-Based Multi-Agent Rollouts

In multi-agent systems, communication can help agents obtain information from other agents and thereby improve coordination or decision-making (Sukhbaatar and Fergus, 2016). Compared with single-agent MBRL, model-based multi-agent learning introduces an additional source of sample complexity, since agents must not only learn the environment dynamics but also obtain reliable estimates of opponent actions during rollouts. In this setting, communication can reduce errors caused by inaccurate opponent predictions, but it is not free and should therefore be used selectively rather than at every rollout step.

A natural way to address this problem is to combine learned opponent models with selective communication during model rollouts. A representative approach following this paradigm is Adaptive Opponent-wise Rollout Policy Optimization (AORPO) (Zhang et al., 2021). AORPO trains opponent policy models using online behavior cloning and employs a self-triggered mechanism that decides when to query other agents based on the prediction error of the opponent models. This mechanism enables agents to switch between model-based opponent predictions and communication within a rollout, thereby balancing model usage and information acquisition.

Despite its effectiveness, AORPO exhibits several limitations. First, communication decisions are determined by model errors estimated during training, which may be suboptimal when applied to rollouts under different states. Second, the self-triggered communication mechanism does not explicitly account for the state-dependent accuracy of the opponent model, potentially leading to suboptimal communication decisions in regions where the learned opponent model remains reliable. Third, AORPO employs a fixed rollout horizon, which does not adapt to the current reliability of the learned environment model. As a result, existing methods still provide limited mechanisms for controlling communication and rollout behavior based on prediction reliability at the current rollout state.

2.4 Summary

In summary, prior work has studied model-based reinforcement learning, opponent modeling in multi-agent settings, uncertainty-aware rollout control, and selective communication. However, these aspects are often addressed separately. In model-based multi-agent rollouts, rollout quality depends not only on the reliability of the learned environment model but also on the accuracy of opponent-action prediction. This motivates a unified framework that uses opponent-model uncertainty to guide communication decisions and dynamics-model uncertainty to control rollout termination.

3 Method

This chapter presents the proposed uncertainty-aware model-based multi-agent reinforcement learning framework, which extends the AORPO framework with uncertainty-aware mechanisms for communication and rollout termination. The central idea is to improve rollout reliability by addressing two sources of error during model-based rollouts: inaccurate opponent-action prediction and unreliable environment-model prediction. To achieve this, opponent-model uncertainty is used to determine when communication is needed, while dynamics-model uncertainty is used to determine when model rollouts should be terminated. The following sections introduce the problem formulation, overall framework, opponent and environment models, uncertainty-aware rollouts, and policy optimization in turn.

3.1 Problem Formulation

We consider a multi-agent reinforcement learning (MARL) problem formulated as a stochastic game. This problem formulation directly follows the framework detailed by [Zhang et al. \(2021\)](#), which represents a standard and widely adopted setting in the MARL literature ([Busoniu et al., 2008](#); [Littman, 1994](#); [Shapley, 1953](#)). Specifically, the stochastic game is defined by the tuple $(\mathcal{S}, \mathcal{A}^1, \dots, \mathcal{A}^n, R_1, \dots, R_n, P_T, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}^i$ the action space of agent $i \in \{1, \dots, n\}$, $R_i : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ the reward function of agent i , $\gamma \in (0, 1)$ the discount factor, and P_T the transition kernel of the environment. The joint action space is given by $\mathcal{A} = \prod_{i=1}^n \mathcal{A}^i$.

At each discrete time step t , the environment is in state $s_t \in \mathcal{S}$. All agents select their actions simultaneously, resulting in a joint action $a_t = (a_t^1, \dots, a_t^n) \in \mathcal{A}$, where $a_t^i \in \mathcal{A}^i$ is the action taken by agent i . The next state is sampled according to the transition dynamics

$$s_{t+1} \sim P_T(\cdot | s_t, a_t). \quad (1)$$

Each agent i receives an individual reward $r_t^i = R_i(s_t, a_t)$.

Each agent follows a stochastic policy $\pi_i(a^i | s)$, which defines a probability distribution over its actions given the current state. We denote the joint policy of all agents except agent i as $\pi_{-i}(a^{-i} | s) = \prod_{j \neq i} \pi_j(a^j | s)$, where a^{-i} represents the actions of all agents except i . Due to the simultaneous execution of actions, an agent cannot observe the actions of other agents before selecting its own action.

From the perspective of a single agent, the environment is inherently non-stationary, as the transition dynamics depend on the evolving policies of the other agents. The objective of each agent i is to find an optimal policy π_i^* that maximizes the expected return (cumulative reward), defined as

$$\max_{\pi_i} \eta_i[\pi_i, \pi_{-i}] = \mathbb{E}_{(s_t, a_t^i, a_t^{-i}) \sim T, \pi_i, \pi_{-i}} \left[\sum_{t=1}^{\infty} \gamma^t R_i(s_t, a_t^i, a_t^{-i}) \right]. \quad (2)$$

We assume that each agent has full access to the global state s_t . In addition to interacting with the environment, an agent i can query another agent $j \neq i$ for its action given an arbitrary state s , i.e., the sample $a^j \sim \pi_j(\cdot | s)$ through communication. Such communication does not require the queried state to correspond to the current environment state. Although communication is generally less expensive than physical environment interaction, it still incurs significant costs and should therefore be minimized.

Under these assumptions, the learning problem involves reducing two distinct types of sample complexities: **environment sample complexity**, which requires learning an accurate model of the transition dynamics P_T ; and **opponent sample complexity**, which involves modeling the policies of other agents π_{-i} without excessive communication ([Zhang et al., 2021](#)).

3.2 Overall Framework

We propose a model-based multi-agent reinforcement learning framework with uncertainty-aware communication and adaptive model rollouts, as illustrated in Fig. 2. In each episode, agent i first interacts with the real environment to collect new trajectories. These data are then used to update both the opponent models and

Algorithm 1 Learning for agent i

```
1: Initialize  $\mathcal{D}_{env}, \mathcal{D}_{sim}$ , opponent models  $\tilde{\pi}_{j,\theta_{o,j}}$ , dynamics model  $\tilde{P}_{T,\theta_T}, \tilde{R}_\psi$ 
2: for each episode do
3:   Reinitialize opponent ensemble models  $\tilde{\pi}_{j,\theta_{o,j}}$ 
4:   Perform rollout in the real environment and save in  $\mathcal{D}_{env,new}$ 
5:   Train  $\tilde{\pi}_{j,\theta_{o,j}}, \tilde{P}_{T,\theta_T}, \tilde{R}_\psi$  on  $\mathcal{D}_{env,new}$ 
6:   for each model rollout do
7:     Sample  $\hat{s}_0$  from  $\mathcal{D}_{env}$ ; initialize  $H_{acc} \leftarrow 0$ 
8:     for each rollout step  $t$  do
9:        $a_t^i \leftarrow \pi_i(\hat{s}_t)$ 
10:      for each opponent  $j$  do
11:        if  $H_{opp}^j(\hat{s}_t) < \lambda_{comm}$  for all action dimensions then
12:           $a_t^j \leftarrow \tilde{\pi}_{j,\theta_{o,j}}(\hat{s}_t)$ 
13:        else
14:           $a_t^j \leftarrow \text{COMM}(\hat{s}_t, j)$ 
15:        end if
16:      end for
17:      Form joint action  $a_t$ 
18:      Predict next state/reward; compute  $H_{dyn}$ ; update  $H_{acc}$ 
19:      if  $H_{dyn} > \lambda_1$  or  $H_{acc} > \lambda_2$  then
20:        Terminate rollout
21:      end if
22:      Save accepted transition  $\rightarrow \mathcal{D}_{sim}$ 
23:    end for
24:  end for
25:  Train the ego policy  $\pi_i$  on  $\mathcal{D}_{sim}$  using SAC
26: end for
```

to generate the action:

$$a_t^j \sim p_{\phi_k}^j(\cdot | \hat{s}_t).$$

This stochastic prediction mechanism allows the rollout to reflect both the learned opponent behavior and the diversity represented by the ensemble.

3.4 Environment Dynamics and Reward Model

With the opponent actions predicted by the opponent models, the ego agent can estimate the future state of the environment. Unlike single-agent settings, state transitions in multi-agent environments depend on the joint actions of all agents. Therefore, we model the environment dynamics as a probabilistic mapping

$$p_\theta(s' | s, a^i, a^{-i}),$$

which takes the current state, the ego action a^i , and the predicted opponent actions a^{-i} as inputs.

We employ an ensemble of probabilistic neural networks to approximate the environment dynamics (Chua et al., 2018). To improve numerical stability, the networks are trained to predict the state difference

$$\Delta s = s_{t+1} - s_t$$

instead of the raw next state. Specifically, each ensemble member m_k (where $k = 1, \dots, K$) outputs the parameters of a Gaussian distribution:

$$p_{\theta_k}(s_{t+1} | s_t, a_t^i, a_t^{-i}) = \mathcal{N}(\mu_{\theta_k}(s_t, a_t^i, a_t^{-i}), \Sigma_{\theta_k}(s_t, a_t^i, a_t^{-i})). \quad (4)$$

Here, μ_{θ_k} and Σ_{θ_k} denote the predicted mean and diagonal covariance of the next-state distribution, respectively.

Algorithm 2 Learning for agent i

```

1:  $\mathcal{D}_{\text{env}}, \mathcal{D}_{\text{sim}} \leftarrow \emptyset$ 
2: Initialize environment ensemble models  $\tilde{P}_{T, \theta_T}$  and reward model  $\tilde{R}_\psi$ 
3: Initialize opponent ensemble models  $\tilde{\pi}_{j, \theta_{0,j}}$  for all  $j \in \{1, \dots, N\} \setminus \{i\}$ 
4: for each episode  $e = 1$  to  $E$  do
5:   Reinitialize opponent ensemble models  $\tilde{\pi}_{j, \theta_{0,j}}$ 
6:   Perform rollout in the real environment and save in  $\mathcal{D}_{\text{env}, \text{new}}$ 
7:   Train  $\tilde{\pi}_{j, \theta_{0,j}}$  on  $\mathcal{D}_{\text{env}, \text{new}}$ 
8:    $\mathcal{D}_{\text{env}} \leftarrow \mathcal{D}_{\text{env}} \cup \mathcal{D}_{\text{env}, \text{new}}$ 
9:   Train  $\tilde{P}_{T, \theta_T}$  and  $\tilde{R}_\psi$  on  $\mathcal{D}_{\text{env}}$ 
10:  for rollout  $r = 1$  to  $R$  do
11:    Sample  $\hat{s}_0$  from  $\mathcal{D}_{\text{env}}$ 
12:    Initialize accumulated uncertainty  $H_{\text{acc}}^{(\ell)} \leftarrow 0$  for all state dimensions  $\ell$ 
13:    for each step  $t = 0$  to  $T - 1$  do
14:       $a_t^i \leftarrow \pi_i(\hat{s}_t)$ 
15:       $\mathbf{a}_t \leftarrow [a_t^i]$ 
16:      for each  $j \neq i$  do
17:        if  $H_{\text{opp}}^{j, (\ell)}(\hat{s}_t) < \lambda_{\text{comm}}^{(\ell)}$  for all  $\ell$  then
18:           $a_t^j \leftarrow \tilde{\pi}_{j, \theta_{0,j}}(\hat{s}_t)$ 
19:        else
20:           $a_t^j \leftarrow \text{COMM}(\hat{s}_t, j)$ 
21:          Save  $(\hat{s}_t, a_t^j)$  in  $\mathcal{D}_{\text{env}, \text{new}}$ 
22:        end if
23:       $\mathbf{a}_t \leftarrow [\mathbf{a}_t, a_t^j]$ 
24:    end for
25:    Predict  $\hat{s}_{t+1}$  and  $\hat{r}_t$  using dynamics and reward models
26:    Compute one-step uncertainty  $H_{\text{dyn}}^{(\ell)}$  and update accumulated uncertainty  $H_{\text{acc}}^{(\ell)}$ 
27:    if there exists  $\ell$  such that  $H_{\text{dyn}}^{(\ell)}(\hat{s}_t, \mathbf{a}_t) > \lambda_1^{(\ell)}$  or  $H_{\text{acc}}^{(\ell)} > \lambda_2^{(\ell)}$  then
28:      break
29:    end if
30:    Save  $(\hat{s}_t, a_t^i, \hat{r}_t, \hat{s}_{t+1})$  in  $\mathcal{D}_{\text{sim}}$ 
31:  end for
32: end for
33: Train the ego policy  $\pi_i$  on  $\mathcal{D}_{\text{sim}}$  using SAC
34: end for

```

Similar to the opponent models, the dynamics ensemble is trained by supervised learning on the real replay buffer $\mathcal{D}_{\text{env}}$. In Algorithm 2, this update is performed after incorporating newly collected real-environment data into $\mathcal{D}_{\text{env}}$ (line 9). We optimize the model parameters by minimizing the negative log-likelihood (NLL) of the observed transitions:

$$\mathcal{L}_{\text{dyn}}(\theta) = -\mathbb{E}_{(s, a^i, a^{-i}, s') \sim \mathcal{D}_{\text{env}}} \left[\sum_{k=1}^K \log p_{\theta_k}(s' | s, a^i, a^{-i}) \right]. \quad (5)$$

This objective allows each ensemble member to learn both the expected transition and the local predictive variance.

During model rollouts, we follow the predictive-distribution construction in InfoProp (Frauenknecht et al., 2025). This yields a single Gaussian predictive distribution for the next state from the ensemble. For a given state-action pair (s_t, a_t^i, a_t^{-i}) , each ensemble member provides a predicted mean $\mu_{\theta_k}(s_t, a_t^i, a_t^{-i})$ and covariance $\Sigma_{\theta_k}(s_t, a_t^i, a_t^{-i})$. These member-wise predictions are fused into a single Gaussian predictive distribution using

covariance intersection with uniform weights:

$$\bar{\Sigma}(s_t, a_t^i, a_t^{-i}) = \left(\frac{1}{K} \sum_{k=1}^K \Sigma_{\theta_k}(s_t, a_t^i, a_t^{-i})^{-1} \right)^{-1}, \quad (6)$$

$$\bar{\mu}(s_t, a_t^i, a_t^{-i}) = \bar{\Sigma}(s_t, a_t^i, a_t^{-i}) \left(\frac{1}{K} \sum_{k=1}^K \Sigma_{\theta_k}(s_t, a_t^i, a_t^{-i})^{-1} \mu_{\theta_k}(s_t, a_t^i, a_t^{-i}) \right). \quad (7)$$

The next state is then generated from the fused predictive distribution as

$$\hat{s}_{t+1} = \bar{\mu}(s_t, a_t^i, a_t^{-i}) + \bar{L}(s_t, a_t^i, a_t^{-i}) w_t, \quad w_t \sim \mathcal{N}(0, I), \quad (8)$$

where $\bar{L}\bar{L}^\top = \bar{\Sigma}$.

In addition to the dynamics model, we maintain a separate deterministic reward model

$$\hat{\mathbf{r}}_t = R_\psi(s_t, a_t^i, a_t^{-i}),$$

where R_ψ is implemented as a multi-layer perceptron that takes the current state, the ego action, and the opponent actions as input, and outputs the immediate rewards of all agents. The reward model is trained on real interaction data using the mean squared error loss:

$$\mathcal{L}_{\text{rew}}(\psi) = \mathbb{E}_{(s, a^i, a^{-i}, \mathbf{r}) \sim \mathcal{D}_{\text{env}}} \left[\|R_\psi(s, a^i, a^{-i}) - \mathbf{r}\|_2^2 \right]. \quad (9)$$

During model rollouts, the learned dynamics model predicts the next state, while the reward model predicts the immediate reward; both operations are included in line 25 of Algorithm 2. The reward of agent i is then used for ego-policy optimization.

3.5 Uncertainty-Aware Adaptive Rollouts

The uncertainty-aware rollout procedure corresponds to lines 10–32 of Algorithm 2. In particular, opponent-model uncertainty is used for communication decisions in lines 17–23, while dynamics-model uncertainty is used for rollout termination in lines 26–29.

To reduce compounding errors in model-based planning, we use an uncertainty-aware rollout mechanism. In multi-agent settings, rollout errors mainly come from two sources. First, the agent may predict the actions of other agents inaccurately. Second, the learned dynamics model may become unreliable as the rollout proceeds. We therefore compute uncertainty for both the opponent models and the dynamics model. The former is used to decide when communication is needed, while the latter is used to decide when model rollouts should be terminated. Our uncertainty-aware rollout design follows the predictive-distribution and adaptive-termination ideas introduced in InfoProp (Frauenknecht et al., 2025).

Opponent-model uncertainty and communication. At each rollout step t , the ego agent must estimate the joint action

$$\mathbf{a}_t = [a_t^i, a_t^{-i}],$$

where the ego action a_t^i is produced by the current policy and the opponent actions a_t^{-i} are predicted by the learned opponent models. To quantify the reliability of these predictions, we construct a fused Gaussian predictive distribution from the opponent ensemble.

For opponent j , let $\mu_{\phi_k}^j(\hat{s}_t)$ and $\Sigma_{\phi_k}^j(\hat{s}_t)$ denote the mean and covariance predicted by ensemble member k at state $\hat{s}_t$. Following the predictive-distribution construction in InfoProp (Frauenknecht et al., 2025), we fuse these predictions using covariance intersection with uniform weights:

$$\bar{\Sigma}_{\text{opp}}^j(\hat{s}_t) = \left(\frac{1}{K} \sum_{k=1}^K \left(\Sigma_{\phi_k}^j(\hat{s}_t) \right)^{-1} \right)^{-1}, \quad (10)$$

$$\bar{\mu}_{\text{opp}}^j(\hat{s}_t) = \bar{\Sigma}_{\text{opp}}^j(\hat{s}_t) \left(\frac{1}{K} \sum_{k=1}^K \left(\Sigma_{\phi_k}^j(\hat{s}_t) \right)^{-1} \mu_{\phi_k}^j(\hat{s}_t) \right). \quad (11)$$

Let $\bar{\sigma}_{\text{opp},j,\ell}^2(\hat{s}_t)$ denote the ℓ -th diagonal element of the fused covariance matrix $\bar{\Sigma}_{\text{opp}}^j(\hat{s}_t)$. We compute the predictive entropy of the ℓ -th action dimension as

$$H_{\text{opp}}^{j,(\ell)}(\hat{s}_t) = \frac{1}{2} \log_2 \left(2\pi e \bar{\sigma}_{\text{opp},j,\ell}^2(\hat{s}_t) \right). \quad (12)$$

Since the communication decision depends only on relative comparisons with thresholds, additive constants do not affect the triggering rule and are therefore omitted when appropriate.

The action of opponent j used in the rollout is then determined by

$$a_t^j = \begin{cases} \tilde{\pi}^j(\hat{s}_t), & \text{if } H_{\text{opp}}^{j,(\ell)}(\hat{s}_t) < \lambda_{\text{comm}}^{(\ell)} \text{ for all } \ell, \\ \text{COMM}(\hat{s}_t, j), & \text{otherwise,} \end{cases} \quad (13)$$

which corresponds to the communication decision in lines 17–23 of Algorithm 2. Here, $\lambda_{\text{comm}}^{(\ell)}$ is the communication threshold for the ℓ -th action dimension, defined as a quantile of the empirical predictive entropy distribution of the opponent model, estimated from environment interaction data. When the predictive entropy remains below the threshold in every action dimension, we rely on the opponent model prediction. Otherwise, the agent communicates with the corresponding opponent and uses the returned action instead. In this way, communication is triggered only when the opponent model is insufficiently reliable at the current rollout state.

Dynamics-model uncertainty and rollout termination. Once the joint action $\mathbf{a}_t = [a_t^i, \hat{a}_t^{-i}]$ is assembled, the dynamics and reward models are used to predict the next transition. Even with accurate action inputs, model-based rollouts may gradually drift into regions where the learned dynamics become unreliable, leading to compounding prediction errors. To control rollout fidelity, we evaluate the uncertainty of the fused predictive distribution defined in Section 3.4.

Let $\bar{\sigma}_{\text{dyn},\ell}^2(\hat{s}_t, \mathbf{a}_t)$ denote the ℓ -th diagonal element of the fused covariance matrix $\bar{\Sigma}_{\text{dyn}}(\hat{s}_t, \mathbf{a}_t)$. The predictive entropy of the ℓ -th state dimension is defined as

$$H_{\text{dyn}}^{(\ell)}(\hat{s}_t, \mathbf{a}_t) = \frac{1}{2} \log_2 \left(2\pi e \bar{\sigma}_{\text{dyn},\ell}^2(\hat{s}_t, \mathbf{a}_t) \right). \quad (14)$$

In our implementation, uncertainty is evaluated independently for each state dimension. The rollout termination is then controlled by comparing the dimension-wise entropy values with thresholds. Moreover, additive constants do not affect the stopping decision and are therefore omitted when appropriate.

The rollout is terminated if either (i) the predictive entropy of any state dimension exceeds a local threshold, or (ii) the accumulated predictive entropy of any state dimension exceeds a global threshold. Specifically, for each state dimension ℓ , let

$$H_{\text{acc}}^{(\ell,t)} = \sum_{\tau=0}^t H_{\text{dyn}}^{(\ell)}(\hat{s}_\tau, \mathbf{a}_\tau) \quad (15)$$

denote the accumulated uncertainty up to step t . Then the rollout stopping time is defined as

$$T_{\text{stop}} = \min \left\{ t \mid \exists \ell \in \{1, \dots, d_s\} \text{ such that } H_{\text{dyn}}^{(\ell)}(\hat{s}_t, \mathbf{a}_t) > \lambda_1^{(\ell)} \text{ or } H_{\text{acc}}^{(\ell,t)} > \lambda_2^{(\ell)} \right\}. \quad (16)$$

In Algorithm 2, the one-step uncertainty is computed and accumulated in line 26, and the corresponding termination condition is checked in lines 27–29. The thresholds λ_1 and λ_2 are defined following the quantile-based strategy in InfoProp (Frauenknecht et al., 2025). Specifically, λ_1 is determined by the accurate quantile ζ_1 of the predictive entropy distribution computed over replay-buffer transitions using the learned dynamics model, while λ_2 is defined using the exceptionally accurate quantile ζ_2 together with a scaling factor ξ . This quantile-based thresholding reflects the typical level of model uncertainty observed on previously seen states and actions. As a result, rollouts are terminated only when predictive uncertainty becomes unusually large.

This keeps model-generated trajectories within regions where the learned dynamics remain reliable and helps reduce error accumulation over long horizons.

Based on the two rules above, we generate simulated transitions from states sampled from the real replay buffer $\mathcal{D}_{\text{env}}$. During rollout, the ego agent follows its current policy. For the other agents, we use the opponent model predictions when they are confident enough, and otherwise use communication to obtain their actions. The dynamics and reward models are then used to predict the next transition. If the uncertainty grows too large, the rollout is stopped early. All accepted transitions are stored in the simulated buffer $\mathcal{D}_{\text{sim}}$ for later policy optimization.

3.6 Policy Optimization

After updating the opponent models and the environment model with real interaction data, we generate simulated transitions for policy learning. Specifically, model rollouts are initialized from states sampled from the real replay buffer $\mathcal{D}_{\text{env}}$. At each rollout step, the ego action is produced by the current policy, while the opponent actions are either predicted by the learned opponent models or obtained through communication when the opponent uncertainty is above the communication threshold. The next state and reward are then predicted by the learned dynamics and reward models. If the rollout uncertainty exceeds the threshold, the rollout is terminated early. In this way, only uncertainty-controlled simulated transitions are added to the simulated buffer $\mathcal{D}_{\text{sim}}$.

We train the ego policy on the simulated buffer $\mathcal{D}_{\text{sim}}$ using Soft Actor-Critic (SAC) (Haarnoja et al., 2018). In Algorithm 2, this policy update is performed in line 33. Since the opponent behaviors have already been incorporated into the simulated transitions through opponent modeling and communication, the policy optimization can be written in the standard single-agent actor-critic form.

Let $\pi_\omega(a^i|s)$ denote the ego policy, and let $Q_{v_1}(s, a^i)$ and $Q_{v_2}(s, a^i)$ be two critic networks. The critics are trained by minimizing the soft Bellman error on simulated transitions:

$$\mathcal{L}_Q(v_m) = \mathbb{E}_{(s, a^i, r, s') \sim \mathcal{D}_{\text{sim}}} \left[(Q_{v_m}(s, a^i) - y(r, s'))^2 \right], \quad m \in \{1, 2\}, \quad (17)$$

where the target value is given by

$$y(r, s') = r + \gamma \mathbb{E}_{a^i \sim \pi_\omega(\cdot|s')} \left[\min_{m \in \{1, 2\}} Q_{\bar{v}_m}(s', a^i) - \alpha \log \pi_\omega(a^i|s') \right]. \quad (18)$$

Here, $Q_{\bar{v}_1}$ and $Q_{\bar{v}_2}$ are the target critics, γ is the discount factor, and α is the temperature parameter for entropy regularization.

The policy is updated by minimizing

$$\mathcal{L}_\pi(\omega) = \mathbb{E}_{s \sim \mathcal{D}_{\text{sim}}, a^i \sim \pi_\omega} \left[\alpha \log \pi_\omega(a^i|s) - \min_{m \in \{1, 2\}} Q_{v_m}(s, a^i) \right]. \quad (19)$$

In our method, the policy update itself follows the standard SAC objective. The main difference is in how the training data are constructed: real interaction data are used to train the opponent and environment models, while policy improvement is performed on simulated transitions generated by the uncertainty-aware rollout procedure (lines 10–32 in Algorithm 2).

4 Results

This chapter evaluates the proposed method on two JaxMARL environments and compares it with the AORPO baseline. We first describe the experimental setup, including environments, evaluation metrics, and implementation details. We then present the main results, analyze the communication–performance trade-off, and study the contributions of the two uncertainty-aware components through ablation experiments. Finally, we discuss the observed advantages and limitations of the proposed method.

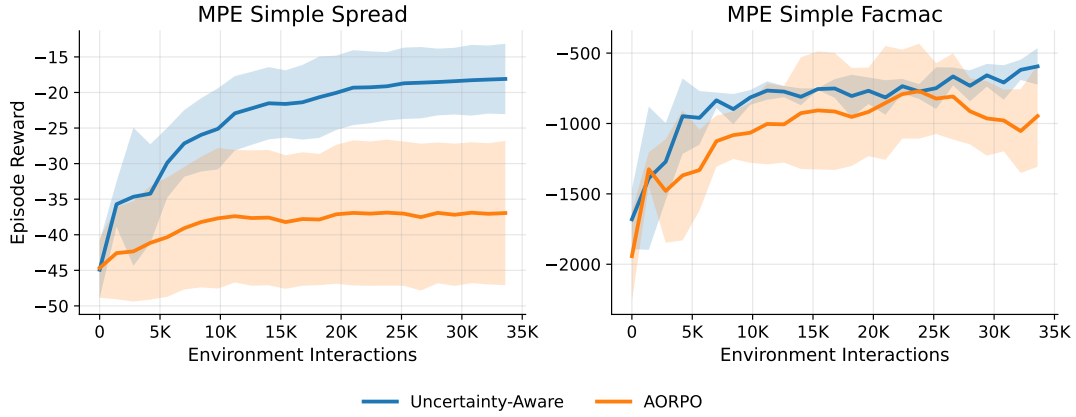


Figure 3. Learning curves on two MPE environments. The shaded areas represent one standard deviation over multiple random seeds.

4.1 Experimental Setup

Environments All environments are taken from JaxMARL. We evaluate the proposed method on two multi-agent environments: `mpe_simple_spread_v3` and `mpe_simple_facmac_3a_v1`.

The `mpe_simple_spread_v3` environment contains three agents and three landmarks. It is a fully cooperative task in which all agents share the same team reward. The objective is to spread the agents over the landmarks so that each landmark is covered by one nearby agent. The reward is defined as the sum of the negative distances between each landmark and its closest agent.

The `mpe_simple_facmac_3a_v1` environment contains three pursuer agents, one prey agent, and two landmarks acting as obstacles. The prey is controlled by a fixed heuristic, rule-based evasion policy, which moves away from nearby pursuers rather than being learned. The goal of the pursuers is to approach and catch the prey while avoiding collisions with the obstacles. A positive reward is given when a pursuer collides with the prey. In addition, to make the reward less sparse, we include a distance-based shaping term defined as the negative distance between the pursuers and the prey.

Baselines. We use Adaptive Opponent-wise Rollout Policy Optimization (AORPO) as the main baseline. It is the most relevant comparison method, since our approach follows the same model-based opponent-modeling framework and mainly differs in the uncertainty-aware communication and adaptive rollout mechanism.

Evaluation Settings and Metrics. Each method is evaluated using 25-step episodes, and we compute the cumulative reward within each evaluation episode. The first metric plots the cumulative reward against training steps, which reflects the learning performance throughout training. The second metric plots the cumulative reward against communication ratio, which is used to evaluate the trade-off between task performance and communication cost.

Implementation Details. All experiments are implemented in JAX. Unless otherwise specified, we use the same model architectures and optimization objectives as described in Section 3. The opponent and dynamics ensemble sizes are set to 5 and 10. The communication threshold is set as the 0.99 quantile of opponent-model uncertainty. For adaptive rollout termination, we adopt the accurate quantile $\zeta_1 = 0.99$, exceptionally accurate quantile $\zeta_2 = 0.01$, and scaling factor $\xi = 30$. More hyperparameter settings are provided in the appendix.

4.2 Main Results

We compare the proposed method with AORPO on the two JaxMARL environments introduced in Section 4.1. Overall, the proposed method achieves better performance on `mpe_simple_spread_v3` and similar performance on `mpe_simple_facmac_3a_v1`. These results indicate that the proposed uncertainty-aware com-

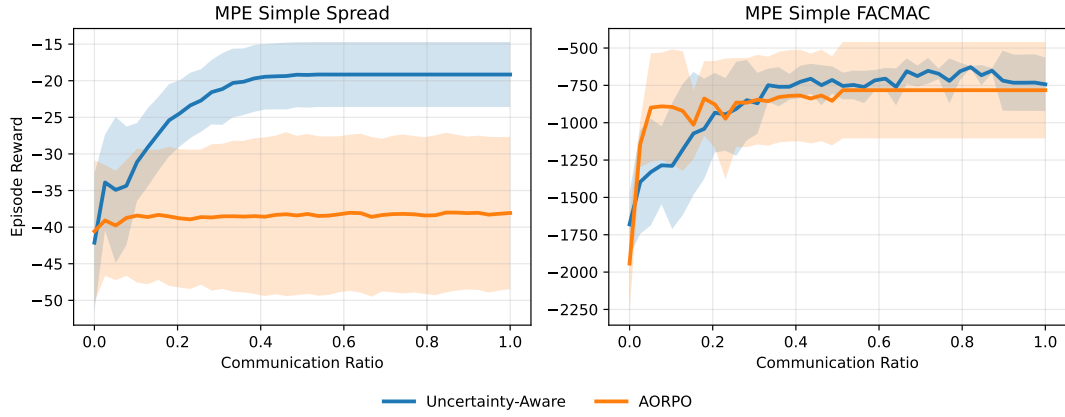


Figure 4. Episode reward as a function of communication ratio. The proposed method achieves higher reward with similar communication in the Spread environment and maintains competitive performance on FACMAC.

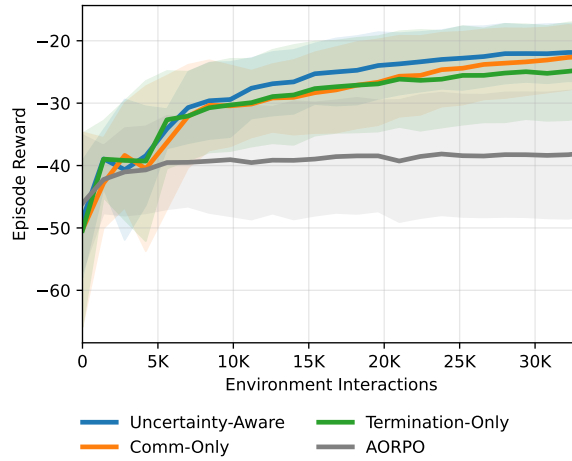


Figure 5. Abation study on the Spread environment. The shaded areas represent one standard deviation over three random seeds.

munication and rollout mechanism can improve performance in the cooperative setting while maintaining comparable performance in the more challenging setting.

Figure 3 shows the learning curves on the two environments. On the Spread environment, the proposed method achieves higher rewards than AORPO during most of training and converges to a better final performance. On the FACMAC environment, the two methods show a similar overall trend, and the proposed method attains a comparable level of performance.

Figure 4 shows the relationship between episode reward and communication ratio. On the Spread environment, the proposed method achieves higher rewards than AORPO under most communication ratios, indicating better communication efficiency. On the FACMAC environment, AORPO attains higher rewards at lower communication ratios in the early stage, while the two methods show a similar overall trade-off later.

4.3 Ablation Study

We further study the Spread environment to understand the contribution of the two main components in the proposed method. Besides the full method, we consider two partial variants: *Comm-Only*, which keeps only uncertainty-aware communication, and *Termination-Only*, which keeps only uncertainty-aware rollout termination. Both variants are compared with the full method and the AORPO baseline.

As shown in Figure 5, the full method performs best throughout training. When either component is removed, the reward drops. However, both partial variants still perform better than AORPO. This means that both uncertainty-aware communication and uncertainty-aware rollout termination are useful. Among the two partial variants, *Comm-Only* performs slightly better than *Termination-Only*. This shows that uncertainty-aware communication helps a bit more in this environment.

4.4 Discussion

The results suggest that the main benefit of the proposed method comes from making communication and rollout decisions based on prediction reliability. In model-based multi-agent learning, errors can come from both inaccurate opponent predictions and unreliable model rollouts. By using uncertainty as a signal, the agent can communicate when the opponent model is less reliable and stop rollouts when the dynamics model becomes less reliable. This helps reduce error accumulation in simulated transitions and makes policy learning more stable.

In the FACMAC environment, the proposed method shows similar performance to AORPO rather than a clear improvement. One possible reason is that the dynamics model is harder to learn in this setting. In FACMAC, the prey keeps moving and the environment changes continuously, which makes the system dynamics more difficult to model accurately. As a result, prediction errors from the learned dynamics model can be larger, and the advantage of using uncertainty-aware communication and rollout termination becomes smaller.

One limitation of the current study is that the experiments are still relatively limited in scope. Although the results on the two environments are encouraging, more experiments on larger or more complex multi-agent tasks are still needed. This would help better understand how well the proposed method generalizes beyond the settings considered in this work.

5 Conclusion

In this thesis, we proposed an uncertainty-aware model-based multi-agent reinforcement learning method that improves both communication decisions and model rollouts. The key idea is to use opponent-model uncertainty to decide when communication with other agents is needed, and to use dynamics-model uncertainty to determine when model rollouts should be terminated. In this way, communication and rollout length are both controlled according to the reliability of model predictions.

The proposed method was evaluated on two JaxMARL environments. The results show that it achieves better performance than AORPO on the cooperative Spread environment and comparable performance on the more challenging FACMAC environment. The ablation study further shows that both uncertainty-aware communication and uncertainty-aware rollout termination contribute to the performance improvement.

One limitation of the current work is that the experiments are conducted on a small number of environments. Future work could evaluate the method on larger or more complex multi-agent tasks. Another possible direction is to improve the dynamics model or uncertainty estimation so that the method can better handle environments with more complex dynamics.

References

- L. Busoniu, R. Babuska, and B. De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- M. P. Deisenroth and C. E. Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, pages 465–472, 2011.
- W. Du and S. Ding. A survey on multi-agent deep reinforcement learning: from the perspective of challenges and applications. *Artificial Intelligence Review*, 53(5):3215–3238, 2020.

- B. Frauenknecht, A. Eisele, D. Subhasish, F. Solowjow, and S. Trimpe. Trust the model where it trusts itself – model-based actor-critic with uncertainty-aware rollout adaptation. In *Proceedings of the International Conference on Machine Learning*, 2024.
- B. Frauenknecht, D. Subhasish, F. Solowjow, and S. Trimpe. On rollouts in model-based reinforcement learning. *arXiv preprint arXiv:2501.16918*, 2025.
- T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.
- H. He, J. Boyd-Graber, K. Kwok, and H. D. III. Opponent modeling in deep reinforcement learning. In *Proceedings of The 33rd International Conference on Machine Learning (ICML)*, 2016.
- P. Hernandez-Leal, M. Kaisers, T. Baarslag, and E. M. de Cote. A survey of learning in multiagent environments: Dealing with non-stationarity. *Autonomous Agents and Multi-Agent Systems*, 31(3):750–798, 2017. doi: 10.1007/s10458-016-9355-7.
- M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- M. L. Littman. Markov games as a framework for multi-agent reinforcement learning. *Proceedings of the Eleventh International Conference on Machine Learning (ICML)*, 1994.
- T. Moerland, J. Broekens, and C. Jonker. A survey of model-based reinforcement learning. *Foundations and Trends in Machine Learning*, 2023.
- R. Raileanu, E. Denton, A. Szlam, and R. Fergus. Modeling others using oneself in multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.
- L. S. Shapley. Stochastic games. *Proceedings of the national academy of sciences*, 39(10):1095–1100, 1953.
- S. Sukhbaatar and R. Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning (ICML)*, pages 216–224. Morgan Kaufmann, 1990.
- E. Talvitie. Model regularization for stable sample rollouts. *UAI*, 2014.
- W. Zhang, X. Wang, J. Shen, and M. Zhou. Model-based multi-agent policy optimization with adaptive opponent-wise rollouts. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI)*, 2021.

A Hyperparameters

Table 1 summarizes the main hyperparameters used in our experiments.

Parameter	Value
Replay buffer size	50,000
Batch size	1024
Training epochs	200
Real-environment steps per epoch	300
Episode length	25
Opponent ensemble size	5
Dynamics ensemble size	10
AORPO Rollout horizon k	6
Communication quantile threshold	0.99
Rollout threshold ζ_1	0.99
Rollout threshold ζ_2	0.01
Scaling factor ξ	30
Dynamics model learning rate	3×10^{-4}
Q-function learning rate	3×10^{-4}
Policy learning rate	1×10^{-2}
Discount factor γ	0.95
Target smoothing τ	0.005
Entropy coefficient α	0.2

Table 1. Main hyperparameters used in the experiments.